MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Privilege Escalation

Unit 11 · Ethical Hacking (10-151-160)

MID-STATE TECHNICAL COLLEGE

Learning Outcomes

By the end of this unit, you will be able to:



Explain escalation methods

Describe common Windows, Linux, and web privilege-escalation techniques and where they fit in the attack lifecycle.



Perform an escalation

Escalate privileges on an authorized lab target and explain the flaw that made it possible.



Explain memory-based attacks

Discuss buffer overflows and process injection at a conceptual level.



Crack hashes and encryption

Use hash- and encryption-cracking tools and explain what makes a scheme crackable.



Detect and defend

Identify the signals of escalation and the controls that prevent it.

LESSON 1

What Privilege Escalation Is



From Foothold to Control

Escalation rarely upgrades your current account — it pivots you to a more powerful one.

- **The adversary seeks higher-level permissions** — moving toward administrator, a service account, or kernel/SYSTEM control.
- **Usually a pivot, not an upgrade** — you move to a different existing account that has more access.
- **Sometimes a brand-new account** — a flaw lets the attacker create one with privileges they never had.

Two directions

System: user → admin → kernel/SYSTEM.

Network: user → network admin → directory (LDAP/AD) admin.

Vertical vs. Horizontal

Two directions of travel, two different goals:

Vertical Escalation

- **Gain more privilege than you have now**
- user → administrator → `SYSTEM` / `root`
- Goal: run privileged code, disable controls, own the host

Horizontal Escalation

- **Gain a peer account's access**
- One user account → a different user account
- Closely related to lateral movement across the network

Horizontal moves often precede a vertical one — reach an account that can, in turn, climb.

Where It Falls in the Attack Lifecycle

Escalation sits at a predictable, defensible chokepoint in an intrusion.

- **After** the attacker has a foothold from exploitation (Units 8–10).
- **Before** lateral movement and acting on objectives.
- Its own ATT&CK tactic — **Privilege Escalation (TA0004)** — overlapping Defense Evasion.



SECURITY FOCUS

Because escalation is a narrow, predictable phase, it is one of the best places to catch an intrusion before it spreads.

Why it matters: Good monitoring here stops a single foothold from becoming full domain compromise.

LESSON 2

Windows & Linux Escalation



Windows Escalation Techniques

Three well-documented Windows paths — recognize and defend, don't memorize exploits.

- **Access Token Manipulation** — tokens decide which user owns a process; attackers duplicate or alter them to act as another identity.
- **Bypassing UAC** — below the highest setting, some trusted programs auto-elevate without a prompt.
- **DLL search-order hijacking** — placing a malicious `.dll` earlier in the load path runs attacker code inside a trusted process.

Reference

MITRE ATT&CK: **T1134** (tokens), **T1548** (UAC), **T1574.001** (DLL hijack).

Linux Escalation Techniques

Enumerate first, then abuse one specific misconfiguration or flaw.

- **Enumeration** — review users, services, cron jobs, and readable files; even an unprivileged user learns a lot.
- **Weak sudo rights** — overly broad entries, or flaws in specific `sudo` versions, grant root commands.
- **Kernel exploits** — a flaw in the kernel itself; the teaching example is **Dirty COW** (`CVE-2016-5195`).



In-class demo

Dirty COW on **Metasploitable2** — a race condition that lets a normal user gain a root account. Watch how long it runs, and why.

LIVE DEMO *Demonstrate the Dirty COW escalation live; have students predict the outcome before it finishes.*

Weak File Permissions & Misconfiguration

Most real-world escalation needs no exploit — just a permission that is too generous.

- **SUID/SGID binaries** — run with the owner's privileges; a root-owned one that runs arbitrary commands is a direct path to root.
- **Writable sensitive files** — editable configs, cron scripts, or service binaries let a user hijack a privileged process.
- **Loose sudo configuration** — permission to run editors, interpreters, or archive tools as root is routinely abused.
- **Exposed credentials and keys** — secrets in scripts and reused SSH keys turn a foothold into access elsewhere.



SECURITY FOCUS

GTFOBins and LOLBAS catalog exactly which trusted binaries can be abused this way — defenders should audit against them.

Why it matters: A single over-permissive file can hand every local user root.

DLL Abuse & Living-off-the-Land

Shared code loaded at runtime becomes an execution and escalation vector.

- **DLLs** are shared modules Windows loads at runtime; control which one loads and your code runs with that program's standing.
- **Trusted binaries (LOLBins)** — `rundll32.exe` and `regsvr32.exe` are abused to run code under a signed, expected process.
- **C2 frameworks** — tools such as Koadic and JSRat ride those same binaries, so defenders watch for odd child processes and callbacks.



Recognition level

We study these to **hunt the pattern** — an unexpected child of a trusted process — not to weaponize it.

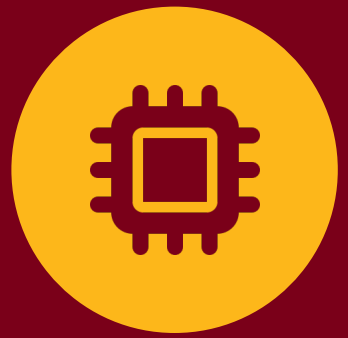
Tools of the Trade

The common toolkit — most automate enumeration or abuse recovered credentials.

Tool	What it does	Phase
Metasploit / Meterpreter	Exploitation framework with built-in escalation and token modules	During / after
Mimikatz	Extracts Windows credentials from memory (needs admin first)	Post-escalation
BloodHound	Maps AD to reveal a path to a more privileged account	Planning
WinPEAS / LinPEAS	Auto-surface likely escalation vectors on a host	Enumeration
GTFOBins / LOLBAS	Catalogs of abusable trusted binaries	Reference

LESSON 3

Buffer Overflows & Process Injection



Ethical Hacking · Privilege Escalation

Buffer Overflows

Writing past a buffer's end can overwrite what controls the program.

- **The flaw** — a program writes more data into a buffer than it holds, spilling into adjacent memory.
- **The impact** — when that memory controls execution, an attacker can redirect it into chosen code.
- **The privilege** — if the vulnerable program runs elevated, the attacker's code inherits those rights.

Modern defenses

DEP, **ASLR**, and **stack canaries** make this far harder — which is why unpatched or poorly built software is the real risk.

Process Injection

Running code inside another live process borrows that process's identity.

How it escalates

- Injected code runs in another process's memory space
- It inherits that process's identity and privileges
- Target a privileged process → gain its rights

Why it hides

- Activity lives inside a trusted process
- Doubles as a defense-evasion technique
- **In-memory** monitoring is the only way to see it

The same reason injection is powerful is why signature-only antivirus misses it.

LESSON 4

Cryptography: Cracking



Ethical Hacking · Privilege Escalation

Hashing vs. Encryption

Once you can read a credential store, the next step is turning protected values back into secrets.

- **Hash** — a one-way fingerprint; cracking means guessing inputs until one matches.
- **Encryption** — reversible with a key; breaking it means recovering or brute-forcing that key.
- **Salting + strong algorithms** — raise the cost enormously, which is why weak or unsalted hashes fall fast.



SECURITY FOCUS

Pass-the-hash: in some Windows cases a recovered hash authenticates directly — no cracking needed.

Why it matters: Protecting the hash itself matters as much as using a strong password.

Cracking Tools

Two standard crackers — both take hashes captured during escalation.

Hashcat

- **GPU-accelerated** password recovery
- Very wide range of hash types
- Dictionary- and rule-based attacks at speed

John the Ripper

- **Flexible**, long-standing cracker
- Auto-detects many hash formats
- Great for learning how cracking modes work

LIVE DEMO *Identify a sample hash type, then launch a dictionary attack and watch it recover a weak password.*

LESSON 5

Detection & Defense



Ethical Hacking · Privilege Escalation

Detecting Privilege Escalation

Escalation is noisy if you are watching the right signals.

- **New privileged accounts** — a fresh admin, or a user added to a privileged group.
- **Odd-hours, long-lived sessions** — privileged activity at unusual times or unusually persistent connections.
- **In-memory behavior** — credential dumping and injection only show up to behavior-based monitoring.
- **Config changes & odd process trees** — UAC changes, new services, or a trusted binary spawning a shell.

Key point

Signature-only antivirus misses in-memory attacks. Detection must watch **behavior**, not just files on disk.

Defensive Measures (Blue Team)

Mostly inexpensive hygiene — the payoff is turning a foothold into a dead end.

- **Separate user and admin accounts** with different credentials; enforce least privilege and patch promptly.
- **Eliminate shared accounts** and password reuse; clean up stale SSH keys and stored secrets.
- **Add strong authentication** — MFA, device fingerprinting, IP/MAC restrictions.
- **Segment and monitor** privileged activity; use behavior-based endpoint protection.

Caveat: Network segmentation may require re-authentication — but if a live session is stolen, that alone will not stop the attacker.

A Finding You Cannot Remediate

Sometimes you cannot close the hole — so you manage it on the record.

SECURITY FOCUS

A legacy app runs only on an out-of-support kernel with a known escalation flaw, and the vendor is gone. You cannot patch it. Convert the exposure into a **documented control requirement**: isolate the host on its own segment, restrict and log all access, add compensating monitoring, and record a formal, time-bound **risk acceptance** signed by the system owner.

Why it matters: An unresolvable finding becomes a managed control with an owner — not a closed ticket and not a silent gap.

Hands-On Lab

Student Kali VMs and authorized lab targets only.



- **Lab 1**
  - **Perform a privilege escalation:** enumerate, escalate by any method, document each step with screenshots, and name the control that would have caught it.
  - **Hash & encryption cracking:** identify hash types, recover weak credentials with Hashcat or John, and compare how salting changes the effort.

Graded on observable outcomes and a clear explanation of how the attack worked — not a screenshot alone.

Key Takeaways



Escalation is a pivot to a more powerful account, not an upgrade of your own.



Most real escalation abuses a misconfiguration or weak permission, not an exploit.



Buffer overflows and injection run attacker code in an elevated context.



Weak or unsalted hashes fall fast; a recovered hash can unlock more access.



Escalation is a detection chokepoint — watch behavior, and document what you cannot fix.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 11 · Privilege Escalation

MID-STATE TECHNICAL COLLEGE